Data Science
Projet – Sujet 1 - Système Intelligent Multi-Modèles pour la Maintenance Prédictive Industrielle

Vous devez soumettre vos livrables sur **MOODLE**, en respectant la date limite fixée par votre enseignant.
Soumettez votre travail personnel. Toute tricherie ne sera pas tolérée et sera sanctionnée.
Ce projet peut être réalisé en binôme.

Cadre Théorique et Rôle du Projet
Machine Learning Supervisé, Deep Learning, Dashboarding et API

Machine Learning Supervisé : Fondements, Méthodes et Rôle dans le Projet

Le Machine Learning supervisé constitue l'un des piliers fondamentaux de l'intelligence artificielle appliquée. Il repose sur un principe simple mais puissant : apprendre automatiquement une fonction de prédiction à partir de données d'entrée (features) associées à une variable cible connue (label). Contrairement à une approche purement statistique descriptive, l'objectif n'est pas seulement de comprendre le passé, mais de généraliser à des situations futures jamais observées.

Dans un contexte industriel, cette capacité prédictive devient stratégique. Les systèmes industriels modernes génèrent en permanence des données issues de capteurs : vibrations, température, pression, vitesse de rotation, modes opératoires, etc. Ces signaux contiennent des patterns subtils qui peuvent annoncer une dégradation progressive ou une panne imminente. Le

Machine Learning supervisé permet d'exploiter ces signaux pour :
- Prédire une panne dans les 24 heures (classification binaire),
- Identifier le type exact de défaillance (classification multi-classe),
- Estimer la durée de vie restante d'un équipement (régression),
- Évaluer le coût probable d'une intervention (régression).

Ainsi, le modèle ne se contente pas de constater un problème ; il devient un outil d'aide à la décision proactive.

Contrairement à une simple corrélation statistique, le Machine Learning :
- apprend automatiquement des relations complexes entre variables,
- capture des interactions non linéaires invisibles à l'œil humain,
- s'adapte à des distributions de données hétérogènes,
- permet une prise de décision automatisée et scalable.

Dans ce projet, vous ne devez pas vous limiter à un seul algorithme. Un des objectifs pédagogiques majeurs est de comprendre que chaque modèle possède ses hypothèses, ses forces et ses limites. Par exemple :
- Une régression logistique peut être robuste et interprétable,

- Un Random Forest peut capturer des non-linéarités complexes,
- Un Gradient Boosting peut offrir de meilleures performances prédictives,
- Un réseau neuronal peut modéliser des interactions plus profondes.

Vous devrez comparer ces approches de manière rigoureuse afin d'identifier :
- Le modèle le plus performant,
- Le modèle le plus stable,
- Le modèle le plus interprétable,
- Le meilleur compromis performance / complexité.

L'objectif n'est pas uniquement d'obtenir le meilleur score, mais de comprendre pourquoi un modèle fonctionne mieux qu'un autre dans un contexte industriel donné.

Deep Learning : Rôle, Intérêt et Analyse Critique

Le Deep Learning représente une évolution du Machine Learning classique, reposant sur des réseaux de neurones artificiels multicouches capables d'apprendre des représentations hiérarchiques des données.

Dans un environnement industriel, les relations entre capteurs et événements de panne peuvent être hautement non linéaires et dépendre de combinaisons complexes de signaux. Les réseaux de neurones (MLP Multi Layer Perceptron) permettent de capturer ces interactions sans nécessiter une ingénierie de features excessive.

Dans ce projet, vous pouvez intégrer :
- Un modèle Deep Learning telque le MLP pour une tâche de classification, ou
- Un modèle Deep Learning pour une tâche de régression.

Cependant, l'intégration du Deep Learning ne doit pas être automatique ou dogmatique. Elle doit être justifiée. Vous devrez analyser :
- Quand un modèle simple est suffisant,
- Quand un modèle plus complexe apporte un gain réel,
- Le compromis biais / variance,
- Le risque d'overfitting,
- Le coût computationnel.

Un réseau neuronal mal paramétré peut sur-apprendre (overfitting) et devenir instable. À l'inverse, un modèle trop simple peut sous-apprendre (underfitting). Votre travail consistera à trouver un équilibre méthodologique.
L'objectif pédagogique est donc de :
1. Comprendre que le Deep Learning n'est pas toujours supérieur.
2. Apprendre à comparer scientifiquement les performances.

Dashboarding et Data Visualization : De la Prédiction à la Décision

Un modèle performant mais incompréhensible n'a que peu de valeur en entreprise. Dans un contexte industriel réel, les décideurs (maintenance managers, ingénieurs, responsables opérationnels) ont besoin d'outils visuels clairs et interactifs.

C'est pourquoi ce projet intègre obligatoirement la conception d'un dashboard décisionnel.
Le dashboard n'est pas un simple affichage graphique. Il doit permettre :

- La visualisation des distributions des capteurs,
- L'analyse des corrélations,
- La comparaison des performances des modèles,
- La simulation d'un scénario machine,
- L'obtention d'une prédiction en temps réel,
- L'analyse des variables les plus influentes.

Vous devrez adopter une approche orientée utilisateur. Posez-vous les questions suivantes :

- Si j'étais responsable maintenance, quelles informations seraient utiles ?
- Comment visualiser clairement le risque de panne ?
- Comment expliquer pourquoi le modèle prend cette décision ?

Le développement de l'interface pourra s'appuyer sur Streamlit, Dash ou tout autre outil jugé pertinent, selon vos choix techniques et votre niveau d'expérience. L'objectif est de transformer un modèle académique en outil opérationnel.

Dans le monde professionnel, un modèle de Machine Learning n'est pas utilisé directement dans un notebook. Il doit être intégré dans une architecture logicielle.

Cette intégration passe par la mise en place d'une API REST.

Une API (Application Programming Interface) permet à d'autres systèmes d'interagir avec votre modèle. Par exemple :

- Une application mobile,
- Un logiciel de supervision industrielle,
- Un ERP,
- Un système de monitoring.

Dans ce projet, vous devrez développer une API REST (via FastAPI ou Flask ou autre) comprenant :

- Un endpoint /predict recevant des données capteurs,
- Un endpoint /health vérifiant l'état du service,
- Optionnel : /model-info fournissant des informations sur le modèle.

L'objectif pédagogique est d'introduire la notion d'industrialisation :

- Séparation modèle / interface,
- Sérialisation du modèle (pickle / joblib),
- Gestion des erreurs,
- Structuration d'un service IA.

Vous devez comprendre qu'un modèle performant n'est qu'une partie du système. Ce qui crée de la valeur en entreprise, c'est son intégration dans un pipeline complet.

Rôle Global du Projet

Ce projet ne consiste pas uniquement à entraîner un modèle. Il s'agit de concevoir un système intelligent complet, comprenant :

1. Préparation des données,
2. Modélisation multi-algorithmes,
3. Évaluation comparative,
4. Interprétabilité,
5. Interface utilisateur,
6. API déployable.

Vous adoptez ainsi une posture de développeur/consultant IA, capable de passer des données brutes à un outil décisionnel industrialisé.

Objectifs Pédagogiques du Projet

En réalisant ce projet, vous apprendrez à :

1. **Analyser et préparer les données**
   Réaliser un audit complet du dataset (valeurs manquantes, outliers, incohérences, fuites de données), définir une stratégie de nettoyage et concevoir un feature engineering pertinent adapté à la problématique métier.
2. **Implémenter des modèles d'apprentissage supervisé**
   Développer et entraîner plusieurs algorithmes de Machine Learning (et éventuellement Deep Learning), en comprenant leurs hypothèses, leurs limites et leurs domaines d'application.
3. **Évaluer rigoureusement les performances des modèles**
   Sélectionner et interpréter des métriques adaptées au problème (RMSE, R^2 pour la régression ; Accuracy, Precision, Recall, F1-score, ROC-AUC pour la classification), et mettre en œuvre des stratégies de validation robustes (validation croisée, analyse des erreurs).
4. **Interpréter les modèles et expliquer les résultats**
   Analyser les contributions des variables (feature importance, SHAP ou équivalent) afin de rendre les modèles compréhensibles et exploitables dans un contexte métier.
5. **Comparer et sélectionner un modèle optimal**
   Comparer plusieurs modèles selon des critères de performance, robustesse, complexité et interprétabilité, et justifier le choix du modèle final en lien avec les contraintes métier.
6. **Concevoir un pipeline Data Science complet**
   Structurer un workflow de bout en bout intégrant la préparation des données, la modélisation, l'évaluation et l'exploitation des résultats dans une logique industrialisable.
7. **Assurer la reproductibilité et la traçabilité des expérimentations**
   Mettre en place des scripts d'exécution (CLI/Makefile), gérer les environnements (requirements.txt / pyproject.toml), versionner les artefacts (modèles, métriques, figures) et suivre les expérimentations (logs, fichiers CSV, MLflow ou équivalent).
8. **Déployer un Proof of Concept (POC) opérationnel**
   Exposer le modèle via une interface exploitable : script CLI (predict.py), notebook interactif ou micro-API (FastAPI/Flask) avec un endpoint de prédiction.
9. **Communiquer les résultats de manière professionnelle**
   Rédiger un rapport synthétique (≤ 6 pages) présentant les données, la méthodologie, les résultats, les limites, les biais et des recommandations actionnables.

10. **Adopter une démarche projet collaborative**
    Travailler en équipe dans une logique d'ingénierie (organisation, répartition des tâches, versioning, qualité du code).
11. **Se positionner comme un consultant en Data Science / IA**
    Présenter une solution de manière claire, structurée et orientée décision, en mettant en avant la valeur métier des résultats.

Compétences visées

Compétences principales évaluées :

- Collecter et préparer des données industrielles
- Concevoir et entraîner plusieurs modèles ML/DL
- Évaluer et comparer des performances
- Prototyper une solution IA complète
- Développer une API d'inférence
- Concevoir un dashboard décisionnel
- Documenter et justifier les choix techniques

Compétences transverses :

- Travail collaboratif
- Rigueur méthodologique
- Capacité d'analyse critique
- Présentation professionnelle

Grille de notation (non certifiante)
(rf. Syllabus du module)

Domaine	Critères	Points
Préparation et qualité des données	Audit, nettoyage, features	4,0
Modèles et évaluation	Baseline, métriques, CV, robustesse	5,0
Reproductibilité & traçabilité	Scripts, env, artefacts, expériences	3,0
Exposition POC	CLI/notebook/app, tests de bon fonctionnement	3,0
Rapport et visualisations	Clarté, limites, biais, reco	3,0
Qualité logicielle	Structure, tests, doc	2,0
	Total	20,0

NB : évaluation pédagogique (non certifiante), destinée à vérifier l'atteinte des acquis du module.

Cahier des Charges du Projet

Description du projet

Dans le cadre de ce projet, vous devez concevoir et développer une plateforme intelligente de maintenance prédictive capable d'exploiter des données issues de capteurs industriels afin d'anticiper les défaillances d'équipements. Le dataset utilisé simule un environnement industriel réaliste dans lequel différentes machines génèrent en continu des mesures physiques telles que la vibration, la température moteur, la pression, la vitesse de rotation (RPM) ou encore le mode de fonctionnement.

L'objectif est de transformer ces signaux bruts en un système d'aide à la décision capable de détecter des patterns annonciateurs de panne. Le système que vous développerez pourra être capable d'effectuer plusieurs tâches prédictives complémentaires, reflétant les besoins réels d'un service maintenance :

- Prédire la probabilité d'une panne dans les 24 heures (classification binaire),
- Identifier le type précis de défaillance (classification multi-classe),
- Estimer la durée de vie restante d'un équipement (régression),
- Évaluer le coût estimé d'une intervention de réparation (régression),
- Etc.

Cependant, dans le cadre de ce projet pédagogique, vous n'êtes pas tenus de réaliser plusieurs tâches de prédiction.
Vous devez choisir une seule tâche de prédiction de votre choix (classification ou régression).
Le jeu de données contient plusieurs variables pouvant servir de variables cibles, ce qui ouvre la possibilité à différentes problématiques prédictives.
Vous sélectionnerez donc une seule tâche et construirez une solution complète et rigoureuse autour de celle-ci.
Points Bonus : Pour plusieurs tâches de prédiction.

Conseil : en autonomie, si vous le souhaitez, vous pourrez par la suite enrichir votre système en développant des composantes complémentaires (autres tâches prédictives, optimisation, monitoring, etc.) afin d'approfondir vos compétences dans le domaine de la maintenance prédictive et de la Data Science industrielle.

Ainsi, votre solution ne sera pas un simple modèle isolé, mais un véritable système intelligent intégrant plusieurs modèles, plusieurs niveaux d'analyse, et plusieurs tâches (si vous le souhaitez).

Au-delà de la performance prédictive, la plateforme devra également intégrer une interface décisionnelle interactive permettant à un utilisateur (par exemple un responsable maintenance) de visualiser les indicateurs clés, simuler des scénarios machine et obtenir des prédictions en temps réel. Le projet adopte donc une approche complète allant de l'analyse des données jusqu'à la mise à disposition opérationnelle du modèle.

Attention : Les visualisations réalisées en tant que Data Scientist pour analyser les données (EDA) et évaluer la performance des quatre modèles testés ne sont pas destinées à être intégrées telles quelles dans le dashboard final.

Ces visuels ont pour objectif d'appuyer votre démarche méthodologique, de justifier vos choix de modélisation et de comparer rigoureusement les performances des différents modèles.

Le dashboard, en revanche, constitue un outil orienté utilisateur métier. Il sera conçu en fin de projet pour permettre à un responsable maintenance de :

- Visualiser les indicateurs clés de performance (KPI),
- Simuler des scénarios en modifiant certains paramètres machine,
- Obtenir des prédictions en temps réel à partir de nouvelles données.

Il est donc essentiel de bien distinguer les visualisations d'analyse scientifique (destinées à votre rapport technique) et l'interface décisionnelle opérationnelle (destinée à l'utilisateur final).

Objectif

L'objectif de ce projet est de concevoir un MVP (Minimum Viable Product) professionnel intégrant l'ensemble des briques fondamentales d'un système d'intelligence artificielle industriel.

Vous devrez combiner plusieurs dimensions complémentaires :

- Une dimension Data Science, incluant le nettoyage, la préparation des données, la modélisation supervisée et l'évaluation rigoureuse des performances.
- Une dimension comparative, imposant l'implémentation de plusieurs algorithmes de Machine Learning et de Deep Learning (optionnel) afin d'analyser leurs performances respectives et d'identifier le modèle le plus pertinent dans ce contexte industriel.
- Une dimension décisionnelle, matérialisée par le développement d'un dashboard interactif permettant d'exploiter les prédictions de manière claire et exploitable par un utilisateur métier.
- Enfin, une dimension d'industrialisation, via la mise en place d'une API REST permettant d'exposer le modèle sous forme de service, comme cela serait fait dans un environnement professionnel réel.

En résumé, ce projet vise à vous placer dans une posture d'ingénieur ou de consultant en intelligence artificielle, capable de concevoir non seulement un modèle performant, mais une solution complète, robuste, explicable et prête à être intégrée dans un système opérationnel.

Dataset

Vous pouvez trouver le dataset sur Kaggle :
https://www.kaggle.com/datasets/tatheerabbas/industrial-machine-predictive-maintenance

Il s'agit de industrial_machine_maintenance.csv dataset qui est partagé avec vous.

Caractéristiques :

- 24 042 enregistrements
- 15 variables
- 4 tâches ML possibles

- Données réalistes simulées

Variables clés :

- vibration_rms
- temperature_motor
- rpm
- pressure_level
- operating_mode
- rul_hours
- failure_within_24h
- failure_type

Consultez Kaggle pour plus d'informations.

Problématiques prédictives possibles

Avec les variables de ce dataset industriel (vibration, température, rpm, pression, mode opératoire, etc.), vous pouvez construire plusieurs problématiques prédictives pertinentes, en classification, en régression, voire en scoring stratégique orienté maintenance et optimisation des coûts.

Tâche prédictive la plus naturelle du dataset :

Prédiction de Panne (Classification Binaire) : Identifier les équipements susceptibles de tomber en panne dans les 24 prochaines heures.

Variable cible : failure_within_24h

Intérêt métier :

- Prioriser les interventions préventives
- Réduire les arrêts non planifiés
- Diminuer les coûts de maintenance corrective
- Améliorer la disponibilité des machines (uptime)

Autres tâches prédictives possibles

Identification du Type de Défaillance (Classification) : Identifier le type précis de panne probable (mécanique, électrique, thermique, etc.).

Variable cible : failure_type

Intérêt métier :

- Préparer les pièces détachées adaptées
- Mobiliser la bonne équipe technique
- Réduire le temps de diagnostic
- Optimiser le stock de maintenance

Estimation de la Durée de Vie Restante (Régression) : Estimer le nombre d'heures restantes avant défaillance.

Variable cible : rul_hours (Remaining Useful Life)

Intérêt métier :

- Planifier la maintenance prédictive
- Optimiser les arrêts planifiés
- Allonger la durée d'exploitation
- Minimiser le risque d'arrêt brutal

Prérequis et exigences principales

- L'analyse doit inclure au minimum 3 modèles différents
- Au moins 1 modèle Deep Learning (optionnel)
- Comparaison quantitative obligatoire
- Dashboard interactif obligatoire
- API fonctionnelle
- Interprétation de l'importance de features
- Interprétation du modèle
- Séparation train/test rigoureuse
- Cross-validation recommandée

Résultats attendus

- Comparaison détaillée des performances
- Choix justifié du meilleur modèle
- Dashboard fonctionnel
- API accessible
- Rapport analytique structuré

Recommandations:

La réussite de ce projet ne repose pas uniquement sur l'implémentation de modèles performants, mais sur une démarche méthodologique rigoureuse, structurée et défendable. L'objectif est que vous soyez capables de passer d'un dataset brut à une solution complète (analyse, modèles, évaluation et dashboard), tout en justifiant vos choix comme dans un contexte professionnel réel. Les recommandations ci-dessous vous guideront tout au long du projet.

Voici les recommandations essentielles pour vous guider tout au long du développement de votre solution :

Commencez par une analyse exploratoire approfondie

Avant toute modélisation, prenez le temps de comprendre vos données : distributions, valeurs extrêmes, cohérence des variables capteurs, présence de valeurs manquantes, éventuelles anomalies et relations entre variables. Une EDA bien menée vous permet de construire une intuition métier : par exemple, identifier quelles plages de vibration ou de température semblent associées à un risque plus élevé. Cette étape sert également à repérer des problèmes invisibles au premier regard (variables constantes, unités incohérentes, valeurs aberrantes, etc.), qui peuvent dégrader fortement les performances.

Vérifiez les distributions des classes
Sur des problèmes industriels (ex. panne dans les 24h), les événements critiques sont souvent rares. Dans ce cas, un modèle peut afficher une accuracy élevée tout en étant mauvais pour détecter les pannes. Vous devez donc analyser le déséquilibre des classes et privilégier des métriques adaptées (Recall, F1, PR-AUC), voire appliquer des stratégies appropriées (stratified split, class_weight, ajustement du seuil de décision). L'objectif est de produire un modèle utile dans un scénario réaliste, où les faux négatifs (panne non détectée) peuvent coûter très cher.

Comment gérer explicitement le déséquilibre des classes si présent ?
Si un déséquilibre significatif est observé, vous devez obligatoirement expérimenter et comparer plusieurs techniques de gestion du déséquilibre.
Analyse préalable: Calculez le ratio de déséquilibre (classe majoritaire vs minoritaire), analysez la matrice de confusion d'un modèle baseline, montrez en quoi l'accuracy est insuffisante dans ce contexte, utilisez des métriques adaptées : Recall (très important en contexte industriel), F1-score, ROC-AUC Receiver Operating Characteristic-Area Under Curve, Precision-Recall AUC (PR AUC) fortement recommandé si déséquilibre important.
Le choix des métriques doit être justifié en fonction du contexte métier (ex. minimiser les faux négatifs pour éviter une panne critique).
Testez des techniques de rééquilibrage (data-level) : implémentez et comparez **au moins deux méthodes parmi :**

- Random Over-Sampling
- SMOTE (Synthetic Minority Over-sampling Technique)
- Random Under-Sampling

Pour chaque méthode : Réentraîner les modèles, comparer les performances avec le baseline, discuter les effets (overfitting, perte d'information, bruit)
Adaptez votre stratégie de validation : Utilisez une validation croisée stratifiée (Stratified K-Fold), justifiez ce choix (préservation des proportions de classes)
Testez des approches au niveau du modèle (recommandé) : En complément du rééchantillonnage, vous pouvez expérimenter :

- Pondération des classes (class_weight = "balanced")
- Apprentissage sensible au coût (cost-sensitive learning)

Ajustez le seuil de décision
Par défaut, un modèle de classification utilise un seuil de 0.5.
Dans un contexte déséquilibré, ce seuil n'est pas optimal.
Vous devez :

- Tester différents seuils
- Identifier un seuil optimisé (ex. maximisation du F1 ou du Recall)
- Analyser l'impact sur les faux positifs / faux négatifs

Comparez et justifiez votre approche finale : Votre analyse doit répondre à :

- Quelle technique améliore réellement la détection ?
- Quel compromis entre précision et rappel est acceptable ?
- Quelle méthode est la plus pertinente dans un contexte métier réel ?

Analysez la corrélation des variables et la redondance des variables
L'étude des corrélations et des relations entre features est importante pour comprendre la structure des données, identifier les variables redondantes et limiter certains risques (multicolinéarité, surinterprétation de variables très liées). Cette analyse vous aide aussi à orienter votre feature engineering : parfois, une variable dérivée (ex. ratio ou combinaison de

capteurs) est plus informative que les variables brutes. L'objectif n'est pas de supprimer mécaniquement des variables, mais de comprendre leur rôle et leur contribution

Implémentez d'abord un modèle baseline simple, puis complexifiez progressivement
Commencez par un modèle simple (e.g. régression logistique, régression linéaire, ou autre) pour établir un point de référence. Ce baseline vous permet de mesurer objectivement le gain apporté par des modèles plus complexes. Ensuite, introduisez progressivement d'autres modèles plus puissants (e.g. Random Forest, Gradient Boosting, Support Vector Machine, MLP, etc.). Cette progression est essentielle : elle vous aide à comprendre ce que chaque famille de modèles apporte, et à éviter de brûler les étapes en allant directement vers du deep learning sans justification.

Ajoutez progressivement des modèles plus complexes :
Comparez systématiquement au moins 3 modèles et choisissez un modèle candidat final
Un attendu clé du projet est la comparaison multi-modèles. Vous devez implémenter au minimum trois modèles (ML classiques et de préférence : au moins un modèle DL (optionnel)) et comparer leurs résultats de manière structurée. À l'issue de cette comparaison, vous devrez sélectionner un modèle candidat final, c'est-à-dire le modèle que vous recommanderiez dans un contexte réel. Ce choix doit être argumenté en prenant en compte plusieurs dimensions : performance, stabilité, interprétabilité, coût de calcul, facilité de déploiement et cohérence métier. Le meilleur modèle n'est pas forcément celui qui a le meilleur score, mais celui qui représente le meilleur compromis pour l'usage visé.
Utilisez des métriques adaptées à chaque tâche (classification ou régression). Présentez vos résultats sous forme de tableaux comparatifs et de visualisations claires. La comparaison doit être structurée et argumentée, et non simplement descriptive.

Évitez le data leakage : un impératif méthodologique
Le data leakage est l'une des erreurs les plus fréquentes en Data Science. Vous devez garantir que les données de test ne contaminent jamais l'entraînement : les étapes de preprocessing (imputation, scaling, encodage) doivent être ajustées uniquement sur le train set puis appliquées au test set. L'usage de pipelines (sklearn.Pipeline, ColumnTransformer) est fortement recommandé pour sécuriser ce point. Un projet présentant une fuite de données compromet la validité des résultats.

Utilisez la validation croisée si pertinent et évaluez la robustesse
Lorsque c'est pertinent, utilisez une validation croisée (cross-validation) afin de rendre vos résultats plus robustes et moins dépendants d'un split unique. Cela permet aussi de mieux comparer les modèles, notamment si les données sont hétérogènes. L'objectif est de montrer que votre modèle généralise réellement et n'est pas performant uniquement sur un découpage favorable.

Optimisez les hyperparamètres de manière raisonnable et justifiée
Vous pouvez utiliser GridSearch ou RandomizedSearch, mais l'optimisation doit être maîtrisée : expliquez votre stratégie, sélectionnez des plages réalistes, et évitez les recherches excessives non justifiées. L'objectif n'est pas de "tuner" au maximum, mais de montrer une démarche rationnelle : améliorer la performance tout en gardant un modèle stable et reproductible.

Analysez les erreurs : comprendre les limites fait partie du travail
Ne vous contentez pas de scores globaux. Vous devez analyser les erreurs : matrices de confusion en classification, résidus en régression, exemples de cas mal prédits, et discussion des raisons possibles (bruit, variables insuffisantes, classes proches, etc.). Cette analyse est essentielle pour formuler des recommandations réalistes (ex. besoins en données supplémentaires, ajout de nouvelles features, ajustement du seuil d'alerte).

Travaillez l'interprétabilité : expliquer est souvent aussi important que prédire
Utilisez des techniques d'importance des variables ou SHAP pour expliquer les décisions du modèle. Dans un contexte industriel, la capacité à expliquer une prédiction est souvent aussi importante que la prédiction elle-même.

- Dans un contexte industriel, un modèle n'est pas jugé uniquement sur sa performance. Il doit aussi être compréhensible et justifiable.
- Un responsable maintenance doit pouvoir répondre à des questions du type : *"*Pourquoi le modèle indique un risque de panne élevé ? Quels signaux capteurs ont déclenché l'alerte ?"
- C'est pourquoi vous devez intégrer des techniques d'explicabilité, telles que l'importance des variables (feature importance) ou, idéalement, SHAP.
- L'objectif est de produire des explications claires, exploitables et cohérentes avec le domaine (ex. vibration et température moteur comme facteurs majeurs).
- Vous devez également être capables d'interpréter ces résultats : distinguer les variables réellement déterminantes, repérer les corrélations trompeuses, et discuter des limites de l'explication (ex. dépendance au modèle, instabilité possible selon les splits).

<u>N.B. : Quand applique-t-on les techniques d'explicabilité (Feature Importance, SHAP) ?</u>
Quand applique-t-on les techniques d'explicabilité (Feature Importance, SHAP) ?
Les techniques d'explicabilité s'appliquent après l'entraînement du modèle, lors de la phase d'évaluation et d'interprétation des prédictions.
En effet, ces méthodes visent à expliquer les décisions d'un modèle déjà entraîné.
Elles permettent de répondre à la question essentielle : Pourquoi le modèle a-t-il produit cette prédiction ?
Sans modèle entraîné et sans prédictions, il n'y a rien à interpréter.

Technique	Quand l'utiliser	Niveau
feature_importances_	Après entraînement	Basique
Permutation Importance	Après évaluation	Recommandé
SHAP	Sur le modèle final sélectionné	Avancé

Feature Importance (importance globale)
La Feature Importance <u>permet d'identifier quelles variables influencent le plus le modèle dans son ensemble.</u>
Elle Analyse le comportement général du modèle pour une vision globale (macro)
e.g : la vibration est la variable la plus influente dans la prédiction des pannes.

Plusieurs techniques possibles :

- <u>Importance native des modèles basés sur les arbres (Random Forest, Gradient Boosting, XGBoost, LightGBM…)</u>
  `model.feature_importances_`

Ces modèles calculent automatiquement une importance basée sur la réduction d'impureté : Gini importance (classification), Gain / réduction de variance (régression)

- Permutation Importance (recommandée)
  `from sklearn.inspection import permutation_importance`
  Méthode agnostique au modèle (fonctionne avec tous les algorithmes).
  Principe :
  1. Mesurer la performance initiale du modèle
  2. Mélanger (permuter) une variable
  3. Recalculer la performance
  4. Observer la perte de performance : Plus la performance chute, plus la variable est importante.

SHAP (explicabilité locale et globale)
SHAP est une méthode avancée d'explicabilité. Elle explique pourquoi une prédiction précise a été faite, pour aller plus loin que la Feature Importance classique.
Elle permet de :
- expliquer une prédiction individuelle (vision locale)
  e.g. Pourquoi cette machine précise a-t-elle une probabilité de panne de 0.82 ?
- obtenir une importance globale des variables :
  Quelles variables influencent le modèle en général ?
- identifier l'impact positif ou négatif de chaque variable sur une prédiction donnée.

Structurez proprement votre code
Organisez votre projet en modules (data preprocessing, modeling, evaluation, API, dashboard). Évitez un notebook monolithique désorganisé.
- Un projet Data Science professionnel ne se résume pas à un notebook unique. Vous devez organiser votre solution comme un mini-produit logiciel, avec une structure claire et maintenable.
- Séparez les responsabilités : préparation des données, entraînement, évaluation, visualisation/dashboard. Un notebook peut être conservé pour l'EDA et l'expérimentation, mais le pipeline final doit pouvoir être exécuté sans dépendre d'un notebook monolithique désorganisé. Cette structuration facilite aussi le travail en équipe et la reproductibilité.

Versionnez votre travail régulièrement avec Git
Cela vous permettra de suivre vos évolutions et d'éviter la perte de travail.
- Le versioning est une bonne pratique incontournable. Utiliser Git vous permet de garder une trace de vos évolutions, de revenir en arrière en cas d'erreur, de travailler efficacement en équipe, et de structurer la collaboration (branches, pull requests, revue de code). Vous devez effectuer des commits réguliers et explicites, reflétant l'avancement réel du projet : nettoyage, ajout d'un modèle, amélioration des features, ajout du dashboard, intégration API, etc. Un dépôt Git propre et vivant constitue un indicateur fort de maturité professionnelle.

Dashboard : indépendant, exploitable, orienté décision
Votre dashboard (Streamlit, Dash/Plotly…) doit être conçu comme un outil décisionnel autonome. Il doit permettre de visualiser les données, d'explorer des indicateurs clés, de comparer les modèles, et d'exécuter des prédictions sur des scénarios saisis par l'utilisateur. L'objectif est de rendre le projet utilisable par un profil métier, même si le code n'est pas exécuté dans un notebook. Le dashboard est donc une composante centrale, et non un simple "bonus visuel".

Industrialisation via API : pour aller plus loin
Testez votre API indépendamment du dashboard
Assurez-vous que vos endpoints fonctionnent correctement avant l'intégration dans l'interface.

- Avant d'intégrer votre modèle dans le dashboard, vous devez valider que votre API fonctionne parfaitement seule. Trop souvent, des projets échouent à la fin car l'API n'a jamais été testée sérieusement.
- Vérifiez donc vos endpoints dès le début : envoyez des requêtes avec Postman, curl, ou un script Python, et assurez-vous que /predict renvoie exactement ce qui est attendu, avec une gestion robuste des erreurs (champs manquants, types incorrects, valeurs incohérentes).
- Le dashboard ne doit être qu'une couche de présentation ; il ne doit pas compenser les faiblesses de l'API. Une API testée en amont vous garantit une intégration fluide, rapide et stable.

Pourquoi industrialiser votre solution en l'exposant via une API REST
Afin de rapprocher le projet d'un usage professionnel réel, votre modèle ne doit pas uniquement fonctionner dans un notebook. Vous devez l'industrialiser en l'exposant via une API REST (FastAPI ou Flask). Cette API jouera le rôle de service d'inférence : elle recevra des données en entrée, appliquera le pipeline de préparation, chargera le modèle entraîné, puis renverra la prédiction.
L'API devra inclure au minimum :

- POST /predict : reçoit un JSON contenant les features (capteurs) et renvoie la prédiction (classe et/ou probabilité pour la classification, valeur pour la régression).
- GET /health : endpoint de santé permettant de vérifier que le service est actif et que le modèle est correctement chargé.
- Gestion des erreurs : validation des entrées (features manquantes, types incorrects, valeurs hors plage), message d'erreur clair et code HTTP approprié.
- Documentation minimale : description du format d'entrée/sortie (ex. exemple de requête et réponse), soit dans le README, soit via la doc auto (Swagger/FastAPI).

Important : le dashboard devra idéalement appeler l'API pour obtenir les prédictions (et non charger directement le modèle), afin de reproduire une architecture réaliste (Front / API / Modèle).

Adoptez une posture professionnelle
Documentez vos choix, justifiez vos décisions, argumentez vos comparaisons. Votre rapport doit refléter une démarche scientifique structurée.

- Ce projet évalue aussi votre capacité à travailler comme un ingénieur IA ou un consultant Data. Votre rapport et votre présentation doivent refléter une démarche scientifique rigoureuse : expliquer vos choix, justifier vos décisions, comparer vos modèles de façon argumentée, et discuter les limites.

- Il ne s'agit pas d'aligner des scores, mais de raconter une analyse structurée : Pourquoi ces features ? Pourquoi ce modèle final ? Quels compromis entre performance, coût et interprétabilité ? Quels risques (biais, surapprentissage, dérive des données) ? Quelles recommandations opérationnelles ?
- Votre valeur ajoutée se mesure dans votre capacité à produire une solution claire, défendable, reproductible, et applicable dans un contexte réel.

En résumé, l'objectif n'est pas seulement d'obtenir le meilleur score possible, mais de démontrer votre capacité à construire une solution complète, robuste, explicable et exploitable. L'industrialisation par API est une extension pour approfondir et valoriser une approche plus proche des pratiques professionnelles.

Exigences Fonctionnelles Détaillées (EF)

EF1 : Acquisition et Préparation des Données
Vous devez mettre en place un pipeline de préparation incluant : nettoyage des valeurs manquantes, encodage des variables catégorielles, normalisation/standardisation si nécessaire, et analyse exploratoire documentée.

EF2 : Modélisation Multi-Algorithmes
Vous devez entraîner et comparer au minimum 4 modèles (classification et/ou régression selon votre choix), incluant des modèles de référence et des modèles plus avancés. Vous devez sélectionner et justifier un modèle candidat final.

EF3 : Système d'Évaluation
Vous devez utiliser des métriques adaptées (classification : Accuracy, Precision, Recall, F1, ROC-AUC ; régression : MAE, RMSE, R^2…) et produire des comparatifs (tableaux et graphes) ainsi qu'une analyse d'erreurs.

EF4 : Dashboard Interactif
Votre interface doit permettre la saisie d'un scénario, afficher la prédiction, comparer les modèles, montrer l'importance des variables et intégrer des graphiques interactifs. Framework conseillé : Streamlit (ou Dash/Plotly) ou tout autre outil jugé pertinent, selon vos choix techniques et votre niveau d'expérience.

EF5 : API
Si vous choisissez l'option industrialisation, vous développerez une API REST exposant au minimum : POST /predict, GET /health, gestion des erreurs, et documentation minimale (README ou Swagger/FastAPI).

Livrables :

- **Vous devez soumettre les livrables sur MOODLE et Git/GitHub**:
  - **Code source de la solution fonctionnelle** (+ documentation technique en annexe)
  - **Rapport du projet (6 pages)**
  - **Support de Présentation du projet**
  - **Vidéo pour présenter votre solution fonctionnelle et faire une démonstration (si vous n'arrivez pas à présenter le projet en classe le dernier jour du module)**
    - **Vidéo enregistrée de démonstration présentant votre solution fonctionnelle (uniquement si vous ne pouvez pas présenter en classe lors de la dernière séance du module)**

Consignes pour la présentation : Une présentation avec démonstration en classe est prévue lors de la dernière séance du module.

- Tous les membres du groupe doivent participer activement.
- L'évaluation sera individuelle.
- En cas d'impossibilité de présenter en classe, vous devrez soumettre une vidéo de présentation et de démonstration complète.

GOOD LUCK!